RAJALAKSHMI ENGINEERING COLLEGE
(An Autonomous Institution, Affiliated to Anna University, Chennai)
Department of Artificial Intelligence and Machine Learning

FINAL PROJECT REPORT

AcadAdvisor

An Agentic AI Based Student Academic Advisor

Regulation-Grounded Advising, Deterministic Academic Tools and an Event-Driven Placement Agent

Submitted in partial fulfilment of the requirements for the degree of
BACHELOR OF TECHNOLOGY
in
ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING

Submitted by
NITHESH KUMAR S (231501114)
LENA VISHAL J (231501083)
LITHESH M (231501084)

Under the guidance of
Dr. M. Soundarya
Department of Artificial Intelligence and Machine Learning

Academic Year 2026 – 2027

AcadAdvisor: An Agentic AI Based Student Academic Advisor

Final Project Report, B.Tech Artificial Intelligence and Machine Learning, Rajalakshmi Engineering College (Anna University)

Nithesh Kumar S (231501114), Lena Vishal J (231501083), Lithesh M (231501084) | Guide: Dr. M. Soundarya

***Abstract*—**Academic regulations decide whether a student may sit an examination, how a CGPA is computed, when an arrear counts, and who qualifies for Honours or a distinction, yet these rules are spread across many clauses of a long regulations document and a 233-page curriculum. Students misread them, faculty advisors answer the same questions repeatedly, and a general-purpose large language model (LLM) can return fluent but unsupported answers and makes arithmetic errors. This report presents AcadAdvisor, an agentic decision-support system for B.Tech Artificial Intelligence and Machine Learning students under Regulation 2023 of Rajalakshmi Engineering College. A LangGraph workflow routes each question through a Memory agent, a Supervisor, a bounded ReAct action controller that chooses one retrieval or one deterministic tool at a time, a Synthesis agent, a Critic/Verifier and a Reflection agent; answers that remain uncertain after two retries are escalated to a counsellor review queue. All arithmetic and eligibility checks are performed by eleven unit-tested Python tools that implement Clauses 4.10, 7, 14.1, 15.5 and 16.2, never by the LLM. Knowledge is grounded in 1,830 page-cited blocks extracted from the official regulations and curriculum PDFs and searched with a hybrid TF-IDF and keyword ranker. A second, event-driven graph monitors a student's Gmail inbox, classifies new mail, and for recruitment drives extracts the company, role and deadline, checks eligibility against the student's own record and drafts a three-day preparation plan. The system runs locally on Ollama (llama3.2:3b) with SQLite persistence and Google sign-in. All 34 automated tests pass; on a ten-question golden set every answer carried page citations and passed verification, the page holding the correct clause was retrieved in the top four for eight questions (mean reciprocal rank 0.658), and the agent invoked deterministic tools for seven of the ten questions. The evaluation also exposes the current limits of retrieval and of the offline verifier, which are discussed together with planned improvements.

***Index Terms*—**Agentic AI, multi-agent systems, retrieval-augmented generation, ReAct, academic advising, human-in-the-loop, LangGraph, Ollama, deterministic tools, event-driven agents.

I. INTRODUCTION

A. Background and Motivation

Every semester, undergraduate students must make decisions that are governed by institutional regulations: whether their attendance in a course allows them to write the end-semester examination, how many additional classes they must attend to recover, what end-semester mark they need to secure a target grade, how an arrear affects their CGPA, and whether they remain eligible for an Honours or Minor degree or for First Class with Distinction. At Rajalakshmi Engineering College these rules are defined in the UG Regulations 2023 [1] and the course structure is defined in the B.Tech AI&ML R2023 Curriculum and Syllabus [2]. The rules are precise, but they are distributed across many clauses, contain exceptions (for example, condonation between 65% and 74% attendance), and interact with one another.

In practice students rely on seniors, informal messages or a faculty counsellor. Manual advising is accurate when the counsellor has time, but it is slow and repetitive, and a misread clause can lead to an invalid registration or a missed condonation window. At the same time, placement-season emails arrive in large numbers, and students frequently miss a drive or apply without meeting its CGPA or arrear criteria. The motivation for this project is to give each student an always-available advisor that answers

from the official documents, computes their personal standing exactly, and keeps a human counsellor in control of uncertain cases.

B. Need for Agentic AI

A single prompt to an LLM is unsuitable for this task for three reasons. First, the model has no knowledge of this institution's regulations, so thresholds and clause numbers may be invented. Second, LLMs are unreliable at arithmetic, and CGPA, attendance and mark-target calculations must be exact. Third, a single response cannot gather more evidence when the first retrieval is insufficient, nor can it recognise that it is uncertain and defer to a human.

Agentic AI addresses these weaknesses by giving the model explicit roles, tools, memory and control flow. Reasoning-and-acting (ReAct) agents interleave reasoning with tool use and observe each result before choosing the next action [3], and retrieval-augmented generation grounds answers in external documents [4]. AcadAdvisor combines both ideas: a controller decides at each step whether to retrieve more policy evidence or to call a deterministic academic tool, a verifier scores how well the answer is grounded, a reflection step rewrites the retrieval strategy when it is not, and a counsellor review queue receives what the system still cannot resolve. The same agentic pattern is applied to inbox monitoring, where the trigger is an external event rather than a chat message.

C. Project Objectives

The objectives of the project are:

- To build a multi-agent advisor that answers regulation and curriculum questions with clause-level and page-level citations drawn only from the official REC documents.
- To implement every calculation (CGPA, attendance standing, classes needed, safe absences, end-semester mark targets, arrear impact, degree classification and Honours eligibility) as deterministic, unit-tested tools that the agent invokes, so that the LLM never performs arithmetic.
- To make retrieval adaptive through a bounded ReAct loop, a groundedness verifier and a reflection step that retries with rewritten queries.
- To keep a human in control by escalating low-confidence answers to a counsellor review queue with approve and reject actions.
- To build an event-driven Gmail agent that classifies new mail and, for recruitment drives, extracts drive details, checks eligibility and produces a preparation plan.
- To deliver an end-to-end product with Google sign-in, per-student persistent memory, personal data upload and a FastAPI interface, running locally on a free model.
- To evaluate the system with automated tests and a golden question set, and to compare it with a basic single-retrieval RAG configuration.

The remainder of this report follows the project life cycle: problem and stakeholder analysis (Sections II and III), the proposed solution, knowledge sources and technology stack (Sections IV to VI), agent architecture, orchestration and RAG design (Sections VII to IX), implementation and evaluation (Sections X and XI), and comparison, innovation and conclusions (Sections XII and XIII).

II. PROBLEM STATEMENT AND BUSINESS CASE

A. Problem Definition

Given a natural-language question from a B.Tech AI&ML R2023 student, such as "I have 68% attendance in NLP; can I write the exam and what should I do?", the problem is to produce (i) a decision grounded in the specific clauses of the official regulations, with page citations, (ii) an exact computation of the student's own standing from their stored grades and attendance, and (iii) concrete next steps,

while recognising when the evidence is insufficient and routing the case to a human counsellor instead of guessing. A parallel problem is to turn each incoming college email into a short, prioritised notification and, for recruitment drives, an eligibility verdict and a preparation plan.

Existing approaches solve this only partially. Reading the regulations directly is authoritative but slow and error-prone. An LLM-only chatbot is fast but unverifiable and cannot compute personal results reliably. A basic RAG assistant improves grounding, but it performs one fixed retrieval, cannot combine several rules with a calculation, and has no mechanism for self-correction or escalation. The gap addressed by this work is an advisor that retrieves adaptively, calculates with tools, verifies itself and keeps a human in the loop.

B. Business and Real-World Context

For an institution, routine advising queries consume counsellor time that could be spent on students who need individual attention. Errors have real costs: a student who does not apply for condonation in time receives an SA (shortage of attendance) grade and must redo the course; a student who misjudges the 45% end-semester floor fails a course despite a good internal score; and a student who misses a placement registration deadline loses an opportunity that may not recur. Because the advisor uses the college's own documents as the single source of truth, an update to the regulations is reflected by re-indexing the PDFs, without retraining any model. Running the model locally through Ollama keeps student data on institutional hardware and removes per-query API costs.

III. USER AND STAKEHOLDER ANALYSIS

A. Target Users

The primary users are B.Tech AI&ML students under Regulation 2023, who need quick, cited answers about attendance, credits, arrears, examinations, CGPA and placements, and who want to plan their own semester. Faculty counsellors are secondary users who review the answers the system flags as uncertain. Administrators configure sign-in, allow-list counsellor accounts and maintain the indexed documents. The academic office benefits indirectly from fewer routine queries.

B. Stakeholder Identification

TABLE I
STAKEHOLDERS AND THEIR INTERESTS

Stakeholder	Primary interest	How AcadAdvisor responds
Students	Correct, quick, personal answers	Cited answers plus exact tool-computed CGPA, attendance and mark targets
Faculty counsellors	Oversight of uncertain cases	Counsellor review queue with approve and reject actions and the full agent trace
Academic office	Fewer routine queries; one source of truth	Answers drawn only from the indexed REC Regulations and Curriculum PDFs
Placement cell	Students register for drives they qualify for	Gmail agent extracts drive details and checks eligibility against each profile
Administrators	Security and privacy of student data	Domain-restricted Google sign-in, allow-listed admins, encrypted Gmail tokens, local SQLite
Academic evaluators	Measurable agentic behaviour	Automated tests, golden-set evaluation and comparison with basic RAG

C. User Requirements

The requirements derived from the stakeholder analysis are summarised in Table II as functional (FR) and non-functional (NFR) requirements.

TABLE II
FUNCTIONAL AND NON-FUNCTIONAL USER REQUIREMENTS

ID	Requirement
FR1	Accept a natural-language question and route it to one of seven intents (placement, exam, attendance, academic rule, curriculum, progress, general).
FR2	Retrieve and cite evidence from the REC UG Regulations 2023 and R2023 AI&ML Curriculum PDFs with page numbers.
FR3	Compute CGPA, attendance standing, classes to 75%, safe absences, mark targets, arrear impact, degree class and Honours eligibility deterministically.
FR4	Verify groundedness, retry retrieval with rewritten queries (at most twice) and escalate unresolved answers to a counsellor.
FR5	Persist each student's profile, conversation history, goals, progress and review decisions across sessions.
FR6	Let students upload grades and attendance as CSV files with row-level validation.
FR7	Monitor a connected Gmail inbox, classify and summarise new mail, and notify the student in-app and by email.
FR8	For recruitment emails, extract company, role, package, deadline and criteria, check eligibility and generate a three-day plan.
FR9	Provide exam preparation plans, curriculum browsing and an evaluation workspace.
NFR1	Correctness: no arithmetic or eligibility decision is delegated to the LLM.
NFR2	Boundedness: at most six controller actions per pass and two reflection retries.
NFR3	Privacy: local model and database; encrypted OAuth credentials; full email bodies not stored.
NFR4	Availability: a deterministic offline fallback when the LLM is unavailable or times out.
NFR5	Traceability: every answer carries an execution trace of thoughts, actions and observations.

IV. PROPOSED AGENTIC AI SOLUTION

A. Overall Solution

AcadAdvisor is a local-first, multi-agent decision-support system. A student signs in with their college Google account and is given a private profile. Questions entered in the Advisor Workbench are processed by the Academic Advisor graph, which recalls the student's context, routes the question, gathers evidence and tool results one action at a time, drafts a cited answer, verifies it and either returns it, retries it or escalates it. Independently, a background worker polls each connected Gmail inbox and runs every new message through the Email Intelligence graph. Both graphs persist their results and traces in SQLite. The overall architecture is shown in Fig. 1 and the advisor workflow in Fig. 2.

B. Key Features

- **Agentic RAG chat:** a ReAct controller chooses retrieval or tool actions based on the latest observations, must retrieve at least once before answering, and is bounded to six actions per pass.
- **Deterministic academic tools:** eleven allow-listed tools wrap pure Python implementations of the regulation clauses, so calculations are exact and testable.
- **Self-correction and HITL:** a Critic/Verifier scores grounding; a Reflection agent rewrites queries for up to two retries; unresolved answers enter the counsellor review queue.
- **CGPA Studio:** grade-sheet editor, what-if simulator, goal-seeking solver for a target CGPA, arrear impact calculator and degree classification tracker.
- **Attendance Planner:** Clause 7 engine with nearest-integer rounding, condonation (65–74%) and appeal (50–64%) zones, classes needed for 75% and a "Can I skip tomorrow?" check.
- **Exam Preparation Studio:** end-semester mark target calculator enforcing the 45% floor, and 3-, 7- and 14-day syllabus-grounded study plans with flashcards.

- **Autonomous Gmail placement agent:** event-driven classification, summarisation, drive extraction, eligibility checking and preparation planning, with notifications to college and personal addresses.
- **Personal data and memory:** CSV upload of grades and attendance, per-student goals, progress and conversation history.

C. Agent-Based Approach

Each competency is assigned to a dedicated agent node with its own prompt and output schema rather than asking one model to play every role. This keeps prompts short, lets each stage be checked independently, and makes failures traceable to a specific node. Agents communicate only through a shared, typed state object maintained by LangGraph, so the order of execution is reproducible and every transition can be recorded in the execution trace. Where a decision has a definitive answer in the regulations (for example, whether 74.5% attendance satisfies Clause 7.1), the agent calls a tool instead of reasoning about it.

V. DATASET AND KNOWLEDGE SOURCES

A. Dataset or Knowledge Source Description

AcadAdvisor is knowledge-driven rather than training-data-driven. No model is fine-tuned; the "dataset" consists of the institution's official documents, a structured course catalogue, student records entered or uploaded by each user, and a golden evaluation set (Table III).

Official documents. The two PDFs in data/pdfs are ingested by core/ingest.py using PyMuPDF. Each page is split at numbered clause headings, unit headings and semester headings; blocks shorter than 80 characters are discarded, and every block keeps its source file and page number so that citations remain stable. The resulting index, data/index/knowledge.json, contains 1,830 blocks with a vocabulary of 7,835 tokens. New PDFs can be added through the /knowledge/upload endpoint, which re-indexes automatically.

Structured curriculum. core/data_curriculum.py encodes the R2023 AI&ML course catalogue for Semesters I to VIII (52 courses with L-T-P-C credits), the professional elective verticals (Advanced AI, Data Science, Cloud and DevOps, Cybersecurity) and syllabus units used by the exam planner.

Student records. Grades and attendance are entered in the app or uploaded as CSV files using the provided templates (grades_template.csv and attendance_template.csv). Demonstration profiles are seeded only in local development mode; no real student data is bundled with the project.

Evaluation data. evaluate.py defines ten golden questions with ground-truth answers and the source page, covering attendance, passing requirements, CGPA, Honours, curriculum, registration and degree classification.

TABLE III
KNOWLEDGE SOURCES AND DATA USED

Source	Content	Size / role
REC UG Regulations 2023 (PDF)	Clauses on registration, attendance, examinations, passing, grading, CGPA and classification	25 pages; 128 indexed blocks
R2023 B.Tech AI&ML Curriculum and Syllabus (PDF)	Semester structure, credits, course syllabi and electives	233 pages; 1,702 indexed blocks
Structured course catalogue	Course codes, L-T-P-C, semesters, elective verticals, units	52 courses, Semesters I–VIII
Student profiles (SQLite)	Grades, attendance, goals, progress, history	Per student; CSV upload supported

Source	Content	Size / role
Sample placement drives	Zoho (8.4 LPA), Amazon AWS (14.5 LPA), Kaar Technologies (7.0 LPA)	Demonstration emails for the placement agent
Golden evaluation set	Questions, ground truth and source page	10 questions across 7 categories

VI. TOOLS AND TECHNOLOGIES USED

Table IV summarises the technology stack. The project runs on Windows 11 with Python 3.11 in a virtual environment and is sized for a 4 GB GPU with 16 GB of RAM.

TABLE IV
TECHNOLOGY STACK

Category	Technology	Purpose
Language	Python 3.11	Agents, tools, services and user interface
Agent orchestration	LangGraph (StateGraph)	Typed shared state, conditional edges and bounded loops; a built-in fallback runner is used if LangGraph is absent
LLM framework	LangChain, langchain-ollama	ChatOllama client at temperature 0 with request timeouts
Language model	Ollama llama3.2:3b (local)	Routing, action selection, synthesis, verification and reflection
Document ingestion	PyMuPDF, pypdf	Page-wise text extraction and clause-aware chunking
Retrieval	Hybrid TF-IDF cosine + keyword overlap (standard library)	Page-cited evidence search without native dependencies
Database	SQLite (11 tables)	Profiles, conversations, reviews, goals, progress, placement emails, Gmail connections and notifications
User interface	Streamlit with built-in OIDC	11-page portal with Google sign-in
Backend API	FastAPI, Uvicorn	Admin REST API with Swagger documentation
Email integration	Google API client, google-auth-oauthlib, APScheduler	Gmail read and send scopes; scheduled background polling
Security	cryptography (Fernet), HMAC token check	Encrypted refresh tokens; constant-time API token comparison
Data and testing	pandas, pytest	CSV validation and 34 automated tests

VII. AGENT ARCHITECTURE

The system is organised in five layers (Fig. 1): a client layer (the Streamlit portal and the admin FastAPI service), the LangGraph orchestration layer hosting the two agent graphs, a knowledge layer, a deterministic tool layer together with the model and integration services, and a SQLite persistence layer.

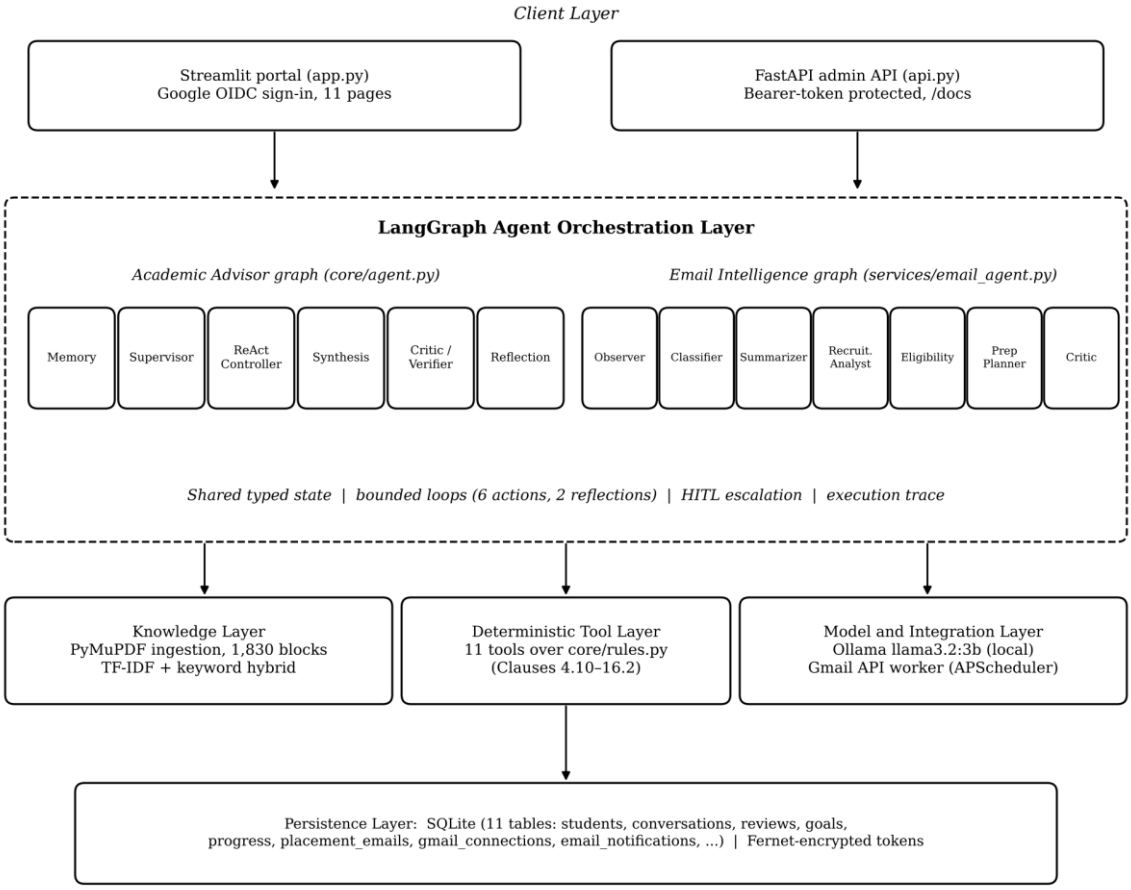


Fig. 1. Layered architecture of AcadAdvisor.

A. Agent Components

TABLE V
AGENT ROLES AND OUTPUTS

Agent	**Role**	**Responsibilities and outputs**
Memory	Context recall	Loads the last turns of conversation, active goals and progress records into working memory
Supervisor	Intent routing	Classifies the question into one of seven intents and records its reasoning (LLM, with a keyword fallback)
ReAct Controller	Action selection	Chooses exactly one action per step: retrieve with a focused query, invoke one allow-listed tool, or finish
Synthesis	Answer drafting	Writes a Decision, Institutional Rationale and Action Steps answer using only retrieved evidence and tool outputs, with clause and page citations
Critic / Verifier	Grounding check	Scores grounding from 0 to 1, lists issues and flags answers below 0.65 or without evidence
Reflection	Self-correction	Uses critic feedback to rewrite up to four retrieval queries and restarts the controller
Email Observer, Classifier, Summarizer	Inbox triage	Capture a new message, assign category and priority, and produce a notification-safe summary
Recruitment Analyst, Eligibility, Preparation Planner	Placement support	Extract drive details, compare criteria with the student profile and build a three-day plan

Agent	Role	Responsibilities and outputs
Email Critic	Output check	Confirms summary, extraction, eligibility and plan exist before notification; otherwise flags for review

B. Tools

The controller may call only the tools listed in core/tools_registry.py (Table VI). Each tool is a thin wrapper over a pure function in core/rules.py and receives the stored student profile, so results are specific to the student without the LLM handling any numbers. Unknown or repeated tool names proposed by the LLM are rejected by the controller.

TABLE VI
DETERMINISTIC TOOL CATALOGUE

Tool	Function	Clause
calc_cgpa	CGPA = Σ(C × GP) / Σ(C), excluding U, SA and WD from the divisor	15.5
goal_seek	Average grade point needed on remaining credits for a target CGPA, with best and worst case	15.5
attendance_summary	Percentage, rounded value, status and clause per course	7.1–7.5
attendance_planning	Classes needed to reach 75% and safe absence margin per course	7.1
skip_evaluation	Whether skipping the next class keeps each course at or above 75%	7.1
degree_classification	First Class with Distinction, First Class or Second Class	16.2
honours_entry	Honours/Minor eligibility: CGPA ≥ 7.50 to Semester III with no arrears	4.10
exam_target	Minimum end-semester mark for a target grade given the CA score, with the 45/100 floor	14.1, 15.1
arrear_impact	CGPA before and after clearing an arrear with a given grade	15.5
student_goals	Active academic goals stored for the student	—
student_progress	Progress tracking records for the student	—

C. Memory

Two kinds of memory are used. Short-term working memory is the LangGraph state for one run: the question, student profile, intent, queries, evidence, tool results, draft, confidence, attempt counter and trace. Long-term memory is the SQLite database: the students table holds each profile with courses and attendance; conversations stores every user and assistant message together with the agent trace; goals and progress hold the student's targets; reviews stores escalated answers and counsellor decisions; and placement_emails, gmail_connections and email_notifications hold the email agent's state. The Memory agent reads the last four turns and the goal and progress records at the start of every run, so follow-up questions are interpreted in context.

D. Reasoning Flow

Within a pass, the controller follows a thought, action and observation cycle. It receives a summary of the evidence and tool results gathered so far and the list of completed actions, and returns a JSON object naming one action and a short thought. After the action executes, its observation (the number of evidence blocks found, or the tool result) is appended to the trace and the controller is invoked again. Evidence from successive retrievals is merged by block identifier and re-ranked, keeping the best eight. The pass ends when the controller chooses finish or reaches six actions. If the LLM is unavailable, the same loop runs with a heuristic policy: retrieve first, then call each tool inferred from the question one at a time, then finish.

E. Agent Interactions

Agents interact by passing structured state rather than free text. The Supervisor's intent and reasoning condition the controller's choices; the controller's evidence and tool results are the only inputs the Synthesis agent may use; the Verifier's issues become the Reflection agent's feedback; and the Reflection agent's rewritten queries force the next controller pass to retrieve before anything else. In the email graph, the Classifier's category decides whether the recruitment branch runs, and the Recruitment Analyst's extracted criteria feed the Eligibility agent.

VIII. AGENT WORKFLOW AND ORCHESTRATION

A. Workflow Design

The advisor workflow is a directed graph built in core/agent.py (Fig. 2). The main path runs from Memory to Supervisor to the controller, which loops on itself until it finishes, then to Synthesis and the Verifier. A conditional edge after the Verifier sends flagged answers to Reflection while fewer than two retries have been used; Reflection returns to the controller. Otherwise the graph ends, and if the answer is still flagged it is written to the counsellor review queue.

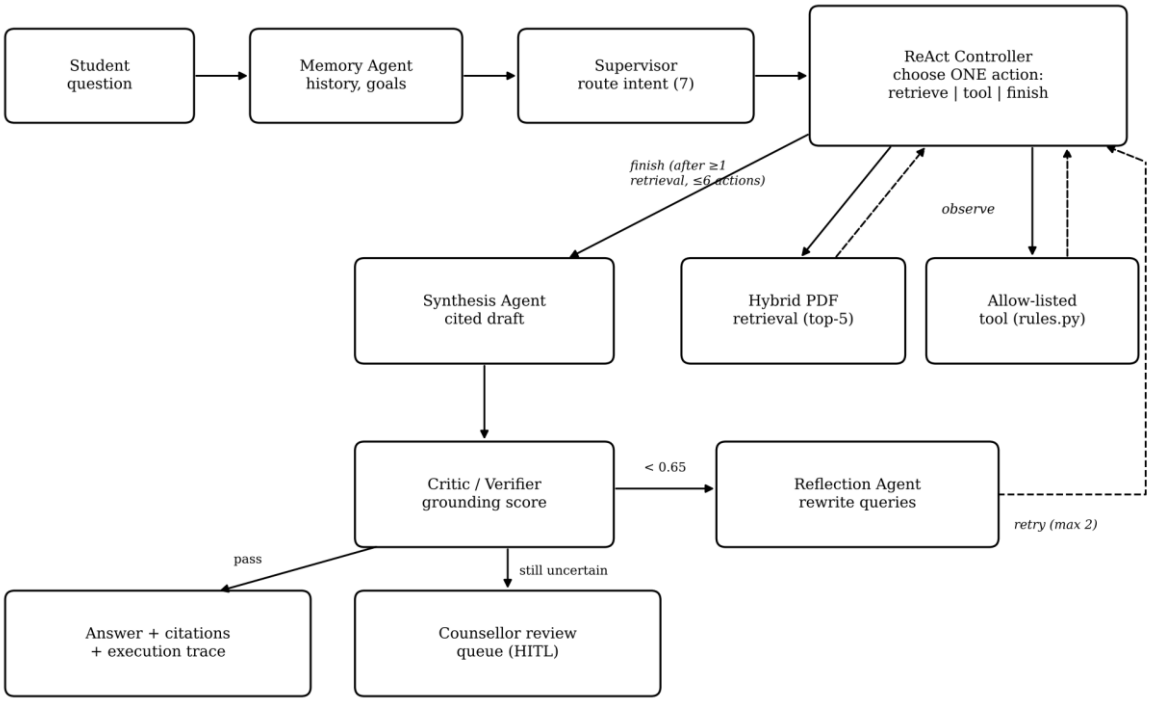


Fig. 2. Academic Advisor workflow with the ReAct loop, reflection retries and HITL escalation.

B. Task Decomposition

TABLE VII
TASK DECOMPOSITION INTO EXECUTION STAGES

No.	Stage	Agent	Stage output
1	Context recall	Memory	Working memory: recent turns, goals, progress
2	Routing	Supervisor	Intent, complexity and reasoning
3	Evidence and calculation	ReAct Controller	Merged, ranked evidence blocks and tool results (≤ 6 actions)
4	Drafting	Synthesis	Cited answer: decision, rationale, action steps
5	Verification	Critic / Verifier	Confidence score, pass or flag, issues

No.	Stage	Agent	Stage output
6	Self-correction	Reflection	Rewritten queries for a fresh pass (≤ 2 retries)
7	Human review	Counsellor	Approve or reject decision stored in reviews

C. Agent Coordination

Coordination rules are enforced in code rather than left to the model: (i) at least one retrieval must occur before the controller may finish; (ii) only allow-listed tools may run, and each at most once per pass; (iii) a repeated query is replaced with an alternative; (iv) a pass stops after six actions; (v) an answer is flagged if confidence is below 0.65, the verifier reports failure, or no evidence was found; (vi) at most two reflection retries are attempted; and (vii) the fallback graph runner aborts after 50 node executions and marks the result for human review. These bounds prevent runaway loops while still allowing the agent to adapt.

D. Event-Driven Email Workflow

The second graph, in services/email_agent.py, is triggered by new mail rather than by a user (Fig. 3). worker.py runs as a separate process and uses APScheduler to poll every connected inbox every GMAIL_POLL_MINUTES minutes (default 2, bounded to 1–10), so monitoring continues when the dashboard is closed. Each message passes through the Observer, Classifier (RECRUITMENT, ACADEMIC, ADMINISTRATIVE or GENERAL, with priority) and Summarizer. Recruitment mail additionally passes through the Recruitment Analyst, Eligibility agent and Preparation Planner. The Critic checks that the required outputs exist, and the notification is shown in-app and sent to the student's college and personal addresses. Messages are deduplicated by Gmail message ID.

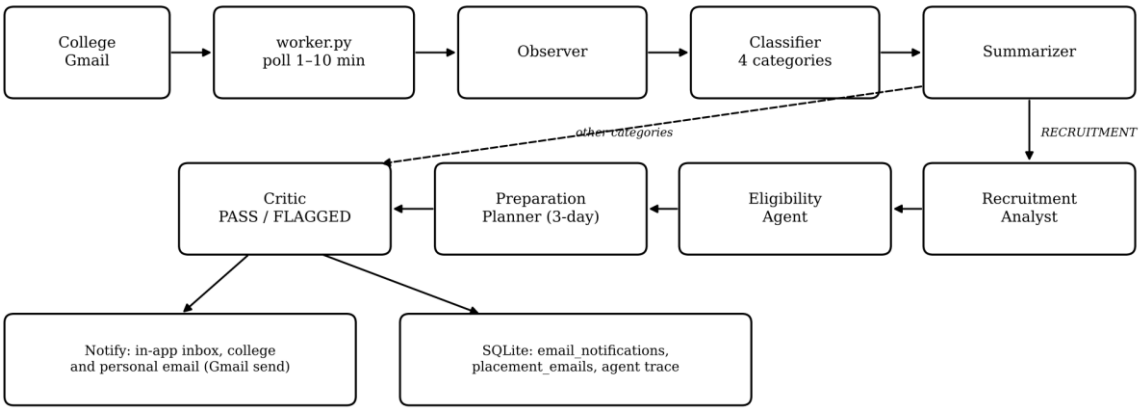


Fig. 3. Event-driven Email Intelligence workflow.

E. Orchestration Implementation

Both graphs are compiled once and reused. The advisor is invoked through answer(question, student_id), which loads the profile and history, runs the graph, stores the conversation with its trace and, when needed, adds a review record. The Streamlit Advisor Workbench displays the trace beneath each answer. The Counsellor Review page and the /reviews, /reviews/{rid}/approve and /reviews/{rid}/reject endpoints let a counsellor resolve escalated answers. LLM calls use a temperature of 0 and timeouts of 18 s (advisor) and 12 s (email) so that a slow model falls back to the deterministic path instead of blocking.

IX. AGENTIC RAG IMPLEMENTATION

A. Retrieval Architecture

Regulation text mixes exact identifiers (clause numbers, course codes such as CS23634, grade letters) with descriptive language. The retriever in core/retrieval.py therefore combines a TF-IDF vector-space model with direct keyword overlap. The tokenizer keeps course codes and dotted clause numbers intact and removes common stop words. Term weights use log-scaled term frequency and smoothed inverse document frequency:

$$w(t, d) = (1 + \ln tf) \cdot [\ \ln((1 + N) / (1 + df(t))) + 1\ ] \quad (1)$$

where N is the number of blocks and df(t) the number of blocks containing t. A block's score combines the cosine similarity of query and block vectors with the fraction of query terms present in the block:

$$score(q, d) = 0.72 \cdot cos(q, d) + 0.28 \cdot |T(q) \cap T(d)| / |T(q)| \quad (2)$$

The overlap term rewards blocks that contain the exact identifiers in the question even when they are rare. The implementation uses only the Python standard library, which avoids native-library problems on Windows and keeps start-up fast.

B. Knowledge Retrieval

Retrieval is agentic because the controller decides when and what to retrieve. It formulates a focused query from the question and the evidence seen so far, may issue further queries if the evidence is insufficient, and cannot finish without at least one retrieval. Each retrieval returns the top five blocks, which are merged with earlier evidence and re-ranked. When the verifier flags an answer, the Reflection agent proposes improved queries (for example, adding the clause topic or "REC 2023 UG regulations") and the next pass begins with a forced retrieval using them.

C. Generation Process

The Synthesis prompt contains the execution plan, the deterministic tool outputs (marked "use exactly, never recalculate"), the top six evidence blocks prefixed with their source and page, and explicit rules: never invent institutional facts, cite clauses and pages in the form [REC-2023-UG-Regulations.pdf p.9 – Clause 7.1], and structure the answer as Decision, Institutional Rationale and Action Steps. Retrieved PDF and email text is treated as data, not instructions, which limits prompt injection. If the LLM returns nothing usable, a grounded fallback composes the answer from clause templates, the evidence and the tool outputs.

D. Retrieval-Quality Evaluation

Retrieval quality was measured on the ten golden questions by checking whether a page that actually contains the answering clause appears among the four cited evidence blocks. The answering pages were located by searching the source PDFs. Results are shown per question in Fig. 4 and summarised in Table VIII.

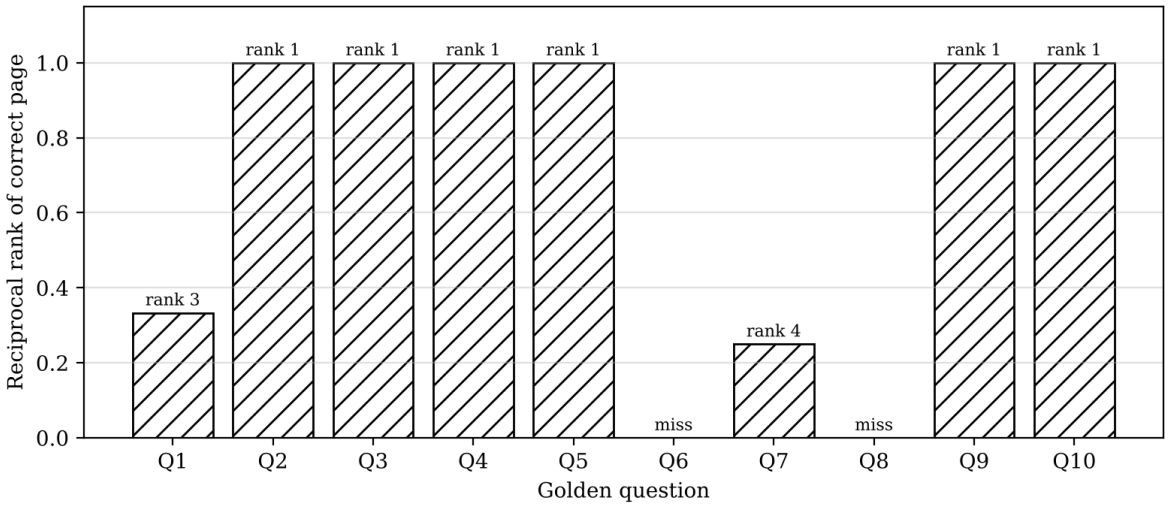


Fig. 4. Reciprocal rank of the first correct page among the four cited blocks, per golden question.

TABLE VIII
RETRIEVAL QUALITY ON THE GOLDEN SET

Configuration	Hit@1	Hit@4	MRR	Mean latency
Basic RAG (one retrieval, top-4)	6 / 10	8 / 10	0.658	33.7 ms
AcadAdvisor agent (offline mode)	6 / 10	8 / 10	0.658	52.5 ms

In offline mode the agent's first retrieval uses the question itself, and because no answer was flagged, reflection was not triggered; its evidence is therefore identical to basic RAG. The two misses were "total credit requirement" (Q6), where blocks mentioning credits in general outranked the curriculum summary page, and "adding or dropping courses" (Q8), where syllabus pages that mention "adding" outranked Clause 6.4. These cases motivate the retrieval improvements listed in Section XIII.

X. SYSTEM IMPLEMENTATION

A. Functional Prototype Screenshots

The prototype is a Streamlit portal with eleven pages: Advisor Workbench, My Academic Data, CGPA and Degree, Attendance Planner, Placement Center, Exam Planner, Curriculum Library, Email Notifications, Counsellor Review, Evaluation Lab, and Architecture and Report. The last three are visible only to allow-listed administrator accounts. Its principal interfaces are shown in the following figures.

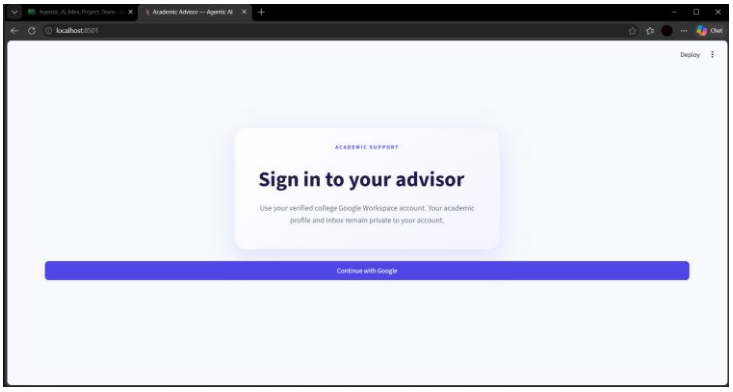


Fig. 5. Google sign-in page and student profile card.

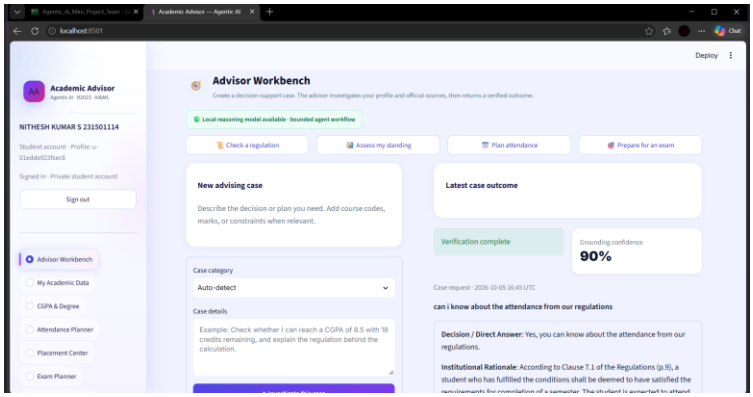


Fig. 6. Advisor Workbench answering a compound attendance question with citations and the agent execution trace.

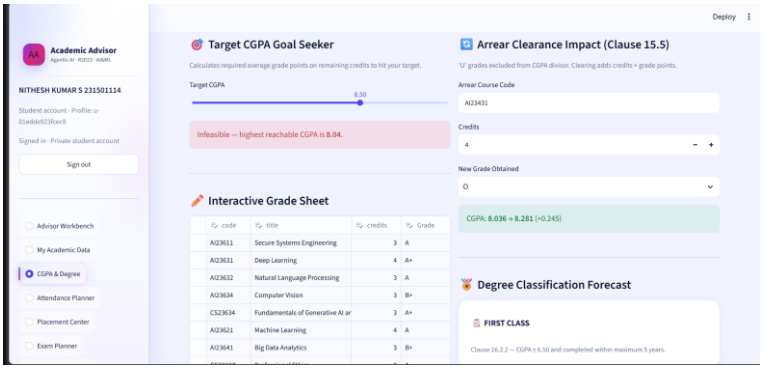


Fig. 7. CGPA Studio with the what-if simulator and goal-seeking solver.

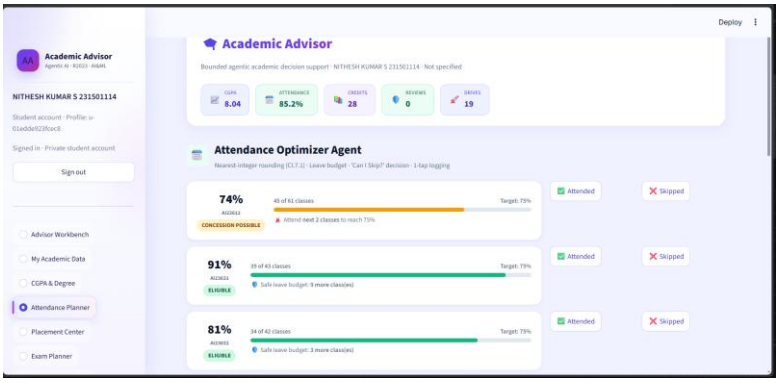


Fig. 8. Attendance Planner showing condonation zone, classes needed for 75% and the skip check.

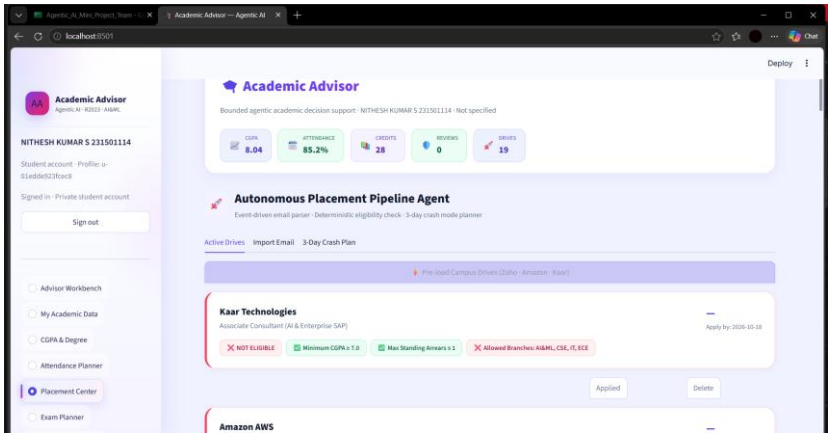


Fig. 9. Placement Center with extracted drive details, eligibility checklist and three-day plan.

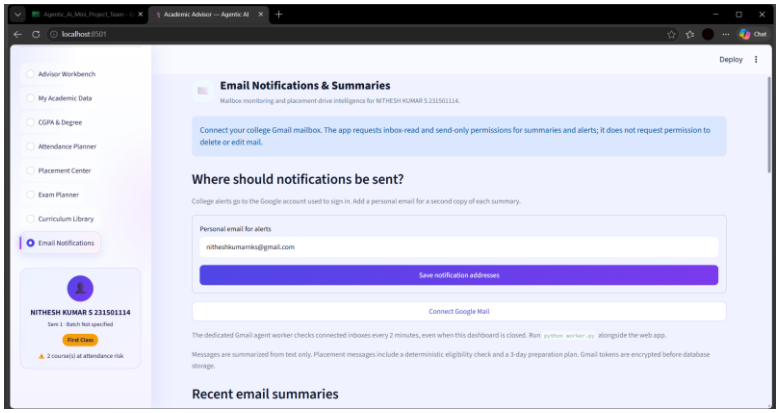


Fig. 10. Counsellor Review queue with approve and reject actions.

B. Source-Code Repository

The source code is organised as follows. Repository URL: [insert GitHub repository link].

TABLE IX
PROJECT STRUCTURE

Path	Contents
app.py	Streamlit portal, navigation and page logic
api.py	FastAPI admin API
worker.py	Background Gmail polling worker (APScheduler)
evaluate.py	Golden-set evaluation harness writing evaluation_results.csv
core/agent.py	Academic Advisor LangGraph: Memory, Supervisor, controller, Synthesis, Verifier, Reflection
core/rules.py, core/tools_registry.py	Regulation rule engine and the eleven-tool catalogue
core/retrieval.py, core/ingest.py	Hybrid TF-IDF retriever and PDF ingestion
core/storage.py	SQLite schema and data access (11 tables)
core/google_auth.py, core/student_data.py	Sign-in identity and roles; CSV parsing and validation
core/data_curriculum.py	Structured R2023 AI&ML course catalogue and electives
services/	email_agent.py, gmail_agent.py, placement.py, exam.py
tests/	34 pytest tests across agent, rules, email agent, sign-in and student data
data/	Source PDFs, knowledge index, CSV templates and database

The application is started with run_app.bat (streamlit run app.py, served at http://localhost:8501), the optional API with run_backend.bat (uvicorn api:app --port 8000, documentation at /docs), and the email worker with run_worker.bat. The API exposes eleven endpoints, grouped in Table X. Every route except /health and the documentation requires an administrator bearer token, and the API is disabled entirely when API_ADMIN_TOKEN is not configured.

TABLE X
API ENDPOINT GROUPS

Group	Endpoints	Count
Health	GET /health	1
Students	GET /students, GET /students/{sid}	2
Advisor	POST /chat	1
Human-in-the-loop	GET /reviews, POST /reviews/{rid}/approve, POST /reviews/{rid}/reject	3
Placement	POST /placement/email, GET /placement/{sid}	2
Exam and knowledge	POST /exam-plan, POST /knowledge/upload	2

XI. EXPERIMENTAL EVALUATION AND RESULTS

A. Evaluation Metrics

The system was evaluated on: (i) correctness of the deterministic tools, measured by automated tests; (ii) citation coverage, the number of page citations attached to each answer; (iii) verifier status and confidence; (iv) a keyword grounding check used by evaluate.py, which tests whether the answer mentions the governing clause or threshold; (v) retrieval Hit@k and mean reciprocal rank (MRR) of the correct page; (vi) tool usage, the number of deterministic tools the agent chose to invoke; and (vii) end-to-end latency.

B. Testing Methodology

Automated tests. The pytest suite contains 34 tests: rule-engine tests (11) covering attendance rounding at 74.5%, the condonation and appeal bands, CGPA exclusion of U grades, the 45% floor and degree classification; agent tests (7) checking that the controller observes each action, cannot finish without evidence, accepts reflection queries and that the verifier cannot pass an answer without evidence; email-agent tests (2); sign-in tests (6) for college-domain restriction, administrator allow-listing and fail-closed behaviour; and data tests (8) for CSV validation, per-owner profile isolation, one-time Gmail OAuth state and Gmail summarisation.

Golden-set evaluation. evaluate.py creates an evaluation student with four Semester VI courses (AI23611, AI23631, AI23632, CS23634) and their attendance, asks the ten golden questions through the full agent, and records latency, confidence, citations, grounding and verifier status. Two runs are reported: an LLM run with llama3.2:3b on Ollama recorded in the project (first three questions), and a complete offline run of all ten questions, in which the LLM is unavailable and the deterministic fallback policy is used. For the retrieval comparison, the same questions were also sent directly to the retriever as a single top-4 query, which is the basic RAG configuration.

C. Performance Results

TABLE XI
AUTOMATED TEST RESULTS

Test module	Tests	Result
test_rules.py	11	Passed
test_agent.py	7	Passed
test_email_agent.py	2	Passed
test_google_auth.py	6	Passed
test_student_data.py	8	Passed
Total	34	34 passed in 0.61 s

TABLE XII
GOLDEN-SET RESULTS (OFFLINE MODE, ALL TEN QUESTIONS)

Q	Category	Tools invoked	Citations	Conf.	Verifier
1	Attendance	attendance_summary, attendance_planning	4	0.99	Verified
2	Attendance (68%, medical)	attendance_summary, attendance_planning	4	0.99	Verified
3	Passing requirement	calc_cgpa, exam_target	4	0.99	Verified
4	CGPA calculation	calc_cgpa	4	0.99	Verified
5	Honours entry	calc_cgpa, honours_entry	4	0.99	Verified
6	Total credits	—	4	0.95	Verified

Q	Category	Tools invoked	Citations	Conf.	Verifier
7	Semester VI courses	—	4	0.95	Verified
8	Add / drop rules	—	4	0.95	Verified
9	Honours credits	calc_cgpa, honours_entry	4	0.99	Verified
10	Distinction	calc_cgpa, degree_classification	4	0.99	Verified

All ten answers carried four page citations and passed verification; nine of ten passed the keyword grounding check. The agent invoked 13 tool calls across seven questions, choosing attendance tools for attendance questions, the exam target tool for the passing-mark question, and the classification and Honours tools where relevant; it correctly invoked no tools for the three purely documentary questions. In the LLM run with llama3.2:3b, the first three questions were answered with confidence 0.99, four citations each and verifier status VERIFIED, with end-to-end latencies of 13.07 s, 9.92 s and 8.62 s on the development laptop. The offline path answers in about 50 ms per question, which shows that the deterministic fallback keeps the system responsive when the model is unavailable.

D. Success-Metric Achievement

TABLE XIII
ACHIEVEMENT OF PROJECT SUCCESS CRITERIA

Success criterion	Observed outcome	Status
Calculations performed by tools, not the LLM	11 tools; 34/34 tests pass, including edge cases	Achieved
Answers cited to official documents	4 page citations on every golden answer	Achieved
Adaptive, bounded agent loop	One action per step, ≤ 6 actions, retrieval required before finishing	Achieved
Context-appropriate tool selection	Tools used on 7/10 questions; none on the 3 documentary ones	Achieved
Self-correction and HITL	Verifier, two reflection retries and counsellor queue implemented and tested	Achieved
Event-driven placement agent	Classification, extraction, eligibility and 3-day plan with notifications	Achieved
Correct page retrieved	Hit@4 = 8/10, MRR = 0.658	Partially achieved
Verifier catches retrieval misses	Offline verifier passed Q6 and Q8 at 0.95	Not yet achieved
Multi-LLM comparison	Protocol defined (llama3.2:3b, qwen2.5:7b, gemma2:9b); full runs pending	Planned

XII. COMPARISON, INNOVATION AND SCALABILITY

A. Comparison with Baseline

The evaluation plan (docs/EVALUATION.md) defines four systems: S0, an LLM with no retrieval; S1, basic RAG with one retrieval; S2, agentic RAG with planning, multi-query retrieval, verification and retry; and S3, the full advisor adding deterministic tools, persistent memory, HITL and the placement and exam agents. Table XIV compares their capabilities.

TABLE XIV
CAPABILITY COMPARISON WITH BASELINES

Capability	S0 LLM-only	S1 Basic RAG	S3 AcadAdvisor
Grounding in REC documents	No	Yes (single query)	Yes (agent-chosen queries)
Page and clause citations	No	Partial	Yes, enforced in prompt
Exact personal calculations	No (LLM arithmetic)	No	Yes, 11 deterministic tools

Capability	S0 LLM-only	S1 Basic RAG	S3 AcadAdvisor
Uses student's stored record	No	No	Yes
Groundedness verification	No	No	Yes
Self-correction (reflection)	No	No	Yes, up to 2 retries
Human-in-the-loop escalation	No	No	Yes, counsellor queue
Conversation and goal memory	No	No	Yes, SQLite
Event-driven inbox agent	No	No	Yes
Works without the LLM	No	Retrieval only	Yes, deterministic fallback

B. Advantages of the Proposed System

- **Correctness:** numerical and eligibility decisions come from tested code that mirrors the clauses, so they are exact and reproducible.
- **Traceability:** every answer carries citations and a step-by-step trace of thoughts, actions and observations.
- **Safety:** uncertain answers go to a counsellor instead of being presented as authoritative.
- **Privacy and cost:** the model and database run locally; Gmail credentials are encrypted and full email bodies are not stored.
- **Resilience:** the deterministic fallback keeps the advisor usable on low-end hardware or when the model is unavailable.

C. Innovation

The contribution of this work is the combination, in a single student-facing product, of several elements that are usually studied separately: a one-action-at-a-time ReAct controller that must retrieve before answering; a strict separation between language reasoning and rule computation through an allow-listed tool catalogue derived from the institution's own regulations; a verifier, reflection and human-escalation chain with hard bounds; and a second, event-driven agent that acts on incoming email without a user prompt and personalises its output using the same student record.

D. Scalability

Functionally, other departments and regulations can be supported by adding their PDFs and course catalogue and, where rules differ, new tools; the graphs do not change. Operationally, the web portal, API and email worker are separate processes and can be deployed and scaled independently. For institution-wide use, SQLite would be replaced by a managed database, polling by Gmail push notifications, and the TF-IDF retriever by a vector store with a hybrid sparse-dense index; each is behind a narrow interface in the current code.

E. Real-World Applicability

AcadAdvisor can serve as a first line of academic advising for students, with counsellors reviewing only the flagged cases. Its limits should be recognised: the regulations and curriculum are those of one programme and one regulation; answers are decision support, not official rulings; Google OAuth for Gmail scopes may require Workspace administrator approval; free hosting can delay background polling; and the SQLite store is a prototype. The security notes in docs/SECURITY.md list the steps required before deployment with real student data, including HTTPS, managed secrets, key rotation, audit logging and institutional approval.

XIII. CONCLUSION AND FUTURE WORK

This report presented AcadAdvisor, an agentic academic advisor for B.Tech AI&ML students under REC Regulation 2023. By combining a bounded ReAct controller, page-cited retrieval over the official regulations and curriculum, eleven deterministic tools for every calculation, a verifier with reflection retries, a counsellor review queue, persistent student memory and an event-driven Gmail placement agent, the system addresses the grounding, arithmetic and oversight weaknesses of single-prompt LLM use. All 34 automated tests pass; on the golden set every answer was cited and verified, the agent selected appropriate tools for seven of ten questions, and the correct page was retrieved in the top four for eight of ten.

Future work includes: (i) improving retrieval with a dense embedding index fused with the current sparse ranker and with query expansion for clause-level questions; (ii) strengthening the offline verifier so that it checks whether the cited block actually contains the answering clause, not only whether citations exist; (iii) running the full S0–S3 comparison across llama3.2:3b, qwen2.5:7b and gemma2:9b with repeated runs for consistency; (iv) extending the golden set to more multi-hop, calculation and out-of-scope trap questions, and correcting its ground truth against the source text (for example, Clause 6.4 specifies add or drop within five working days); (v) adding semester planning with a planner and critic that respect prerequisites and credit limits, with counsellor approval; and (vi) moving to a managed database and Gmail push notifications for institutional deployment.

REFERENCES

[1] Rajalakshmi Engineering College, "Regulations 2023 for B.E./B.Tech. Degree Programmes (Choice Based Credit System)," Chennai, India, 2023.

[2] Rajalakshmi Engineering College, Department of AI&ML, "B.Tech. Artificial Intelligence and Machine Learning, Regulation 2023: Curriculum and Syllabus," Chennai, India, 2023.

[3] S. Yao et al., "ReAct: Synergizing reasoning and acting in language models," in Proc. Int. Conf. Learning Representations (ICLR), 2023.

[4] P. Lewis et al., "Retrieval-augmented generation for knowledge-intensive NLP tasks," in Proc. Adv. Neural Inf. Process. Syst. (NeurIPS), vol. 33, 2020, pp. 9459–9474.

[5] N. Shinn, F. Cassano, A. Gopinath, K. Narasimhan, and S. Yao, "Reflexion: Language agents with verbal reinforcement learning," in Proc. Adv. Neural Inf. Process. Syst. (NeurIPS), vol. 36, 2023.

[6] A. Asai, Z. Wu, Y. Wang, A. Sil, and H. Hajishirzi, "Self-RAG: Learning to retrieve, generate, and critique through self-reflection," in Proc. Int. Conf. Learning Representations (ICLR), 2024.

[7] G. Salton and C. Buckley, "Term-weighting approaches in automatic text retrieval," Inf. Process. Manage., vol. 24, no. 5, pp. 513–523, 1988.

[8] T. Schick et al., "Toolformer: Language models can teach themselves to use tools," in Proc. Adv. Neural Inf. Process. Syst. (NeurIPS), vol. 36, 2023.

[9] LangChain, "LangGraph documentation." [Online]. Available: https://langchain-ai.github.io/langgraph/

[10] Ollama, "Ollama documentation." [Online]. Available: https://ollama.com/

[11] Streamlit, "Streamlit documentation." [Online]. Available: https://docs.streamlit.io/

[12] S. Ramírez, "FastAPI framework documentation." [Online]. Available: https://fastapi.tiangolo.com/

[13] Google, "Gmail API overview." [Online]. Available: https://developers.google.com/gmail/api